Formularios reactivos: alta de productos de inventario

App **inventario-app** · Ionic 9 + Angular 22 standalone · Reactive Forms + Signals + Capacitor Preferences

Jorge González Gómez · Desarrollo de Interfaces (0488) · 2º DAM · Universidad Nebrija

1. Estructura del proyecto

Misma base que mis prácticas anteriores (Ionic standalone, sin NgModules). Separé el modelo, los validadores y el guardado en su propio fichero para que la página solo se encargue de la vista:

```
inventario-app/src/app/
|-- models/producto.model.ts              CATEGORIAS, tipo Categoria e interfaces Producto
|-- validators/producto.validators.ts     3 validadores personalizados (+ .spec.ts con sus tests)
|-- services/inventario.service.ts        "petición" simulada + guardado local (Preferences)
`-- pages/nuevo-producto/                 la vista: .ts (lógica) + .html (plantilla) + .scss
```

2. El formulario reactivo

Todo el formulario se define en TypeScript con `FormBuilder`; en el HTML solo se enlaza con `[formGroup]` y `formControlName`:

```
protected readonly form = this.fb.nonNullable.group({
  sku:         ['',   [Validators.required, skuValidator(12)]],
  nombre:      ['',   [Validators.required, noSoloEspaciosValidator, Validators.maxLength(60)]],
  categoria:   ['' as Categoria | '', Validators.required],
  stockMinimo: [null as number | null, [Validators.required, enteroPositivoValidator]],
});
```

Campo	Componente Ionic	Validación
Código de barras (SKU)	`<ion-input>` con `maxlength=12` y contador `0 / 12`	Obligatorio + `skuValidator(12)`: solo letras/números y exactamente 12
Nombre	`<ion-input>`	Obligatorio + `noSoloEspaciosValidator` (rechaza `"  "`) + máx. 60
Categoría	`<ion-select interface="action-sheet">`: Electrónica, Herramientas, Consumibles	Obligatorio
Stock mínimo	`<ion-input type="number" inputmode="numeric">` → teclado numérico nativo	Obligatorio + `enteroPositivoValidator`: > 0 y entero (no hay media unidad)

3. Validadores personalizados y mensajes de error

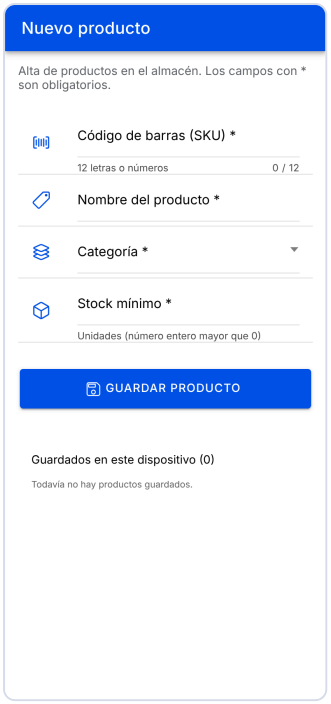
Un validador es una función que recibe el control y devuelve `null` si está bien o un objeto con el error. Si el campo está vacío devuelven `null` y lo deja a `Validators.required`, así cada uno comprueba una sola cosa. `skuValidator` es una **fábrica** (recibe la longitud y devuelve el validador), igual que `Validators.minLength(3)`; además devuelve datos en el error para el mensaje:

```
if (!/^[A-Za-z0-9]+$/.test(valor)) return { skuCaracteres: true };
if (valor.length !== longitud)    return { skuLongitud: { requerida: longitud, actual: valor.length } };
```

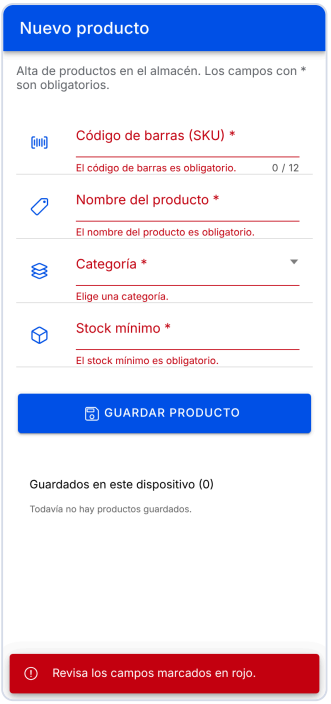
El método `mensajeError(campo)` mira qué error tiene cada control y devuelve el texto que se pasa a `[errorText]`. Ionic solo lo enseña cuando el campo tiene las clases `ion-invalid` e `ion-touched`, es decir, después de que el operario haya pasado por él.

> **Cambio respecto al ejemplo del dossier:** allí el botón se desactiva con `registroForm.invalid`, así que nunca se llega a ejecutar `markAllAsTouched()`. Aquí el botón solo se bloquea mientras se guarda: si el formulario es inválido, al pulsar se marcan todos los campos, se ven todos los errores a la vez y sale un toast rojo.

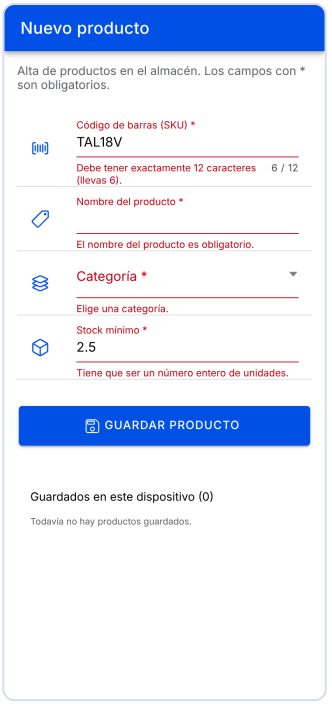
4. Funcionamiento (capturas en vista móvil, 390 × 844)



Vista inicial. Iconos, etiquetas flotantes, texto de ayuda y contador del SKU.



Enviar vacío. `markAllAsTouched()` enseña los 4 `errorText` y salta el toast rojo.



Validadores propios. SKU de 6 → "llevas 6"; nombre solo con espacios; stock 2.5 → no entero.



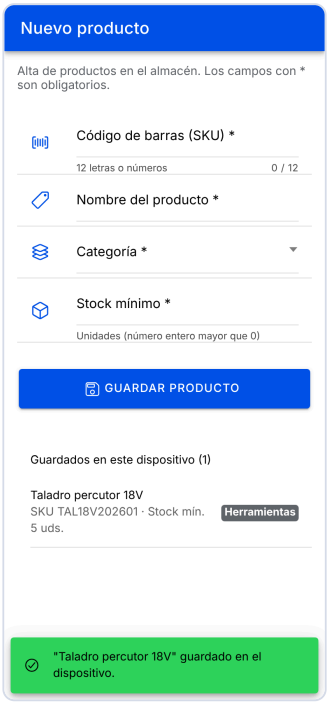
ion-select. Las 3 categorías en un action sheet (cómodo con el pulgar). Stock 0 → "mayor que 0".



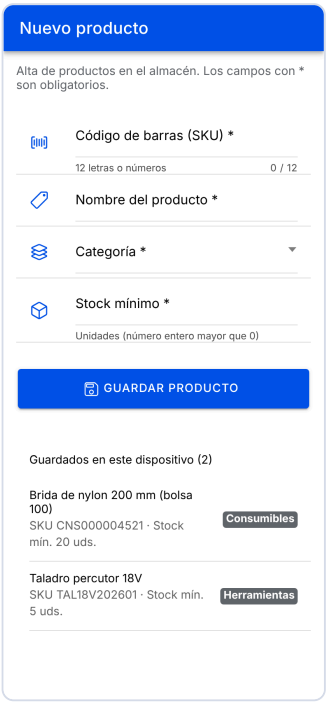
Formulario válido. SKU escrito en minúsculas: se guarda en mayúsculas.



Guardando. `guardando()` = true: campos desactivados, spinner en el botón y barra de progreso.



Guardado. `<ion-toast>` verde, el form se limpia con `reset()` y el producto aparece en la lista.



Tras recargar (F5). Los 2 productos siguen ahí: están en el almacenamiento del dispositivo.

5. Estado del guardado con un Signal<boolean>

Un solo signal, `guardando`, controla todo lo que cambia mientras se guarda. Un `effect()` lo sincroniza con el formulario, y la plantilla lo lee para el botón, el spinner y la barra de progreso:

```
protected readonly guardando = signal<boolean>(false);

// en el constructor: el signal manda sobre el formulario
effect(() => {
  if (this.guardando()) this.form.disable();
  else this.form.enable();
});
```

```
<!-- plantilla -->
<ion-button expand="block" type="submit" [disabled]="guardando()">
  @if (guardando()) { <ion-spinner name="crescent"></ion-spinner> Guardando… }
  @else { Guardar producto }
</ion-button>
```

```
const datos = this.form.getRawValue();   // antes de desactivar: un form desactivado no da su .value
this.guardando.set(true);                 // → form desactivado + loader
try {
  const producto = await this.inventario.guardar({ ...datos, sku: datos.sku.toUpperCase(), ... });
  this.form.reset();
  this.abrirToast(`"${producto.nombre}" guardado en el dispositivo.`, 'success');
} catch { this.abrirToast('No se pudo guardar el producto...', 'danger'); }
finally { this.guardando.set(false); }   // → se reactiva pase lo que pase
```

El toast es un `<ion-toast>` en la plantilla controlado también con signals: `[isOpen]="toastAbierto()"`, `[message]`, `[color]` y `(didDismiss)` para cerrarlo.

6. Guardado local (Offline First)

Como en el punto 1.4 del dossier, el destino es el propio dispositivo: `InventarioService` espera 1,5 s (simula la petición), añade `id` y `fechaAlta`, y guarda la lista como JSON con **Capacitor Preferences** (en Android/iOS va al almacenamiento nativo; en `ionic serve`, a `localStorage`). La lista se expone en solo lectura con `asReadonly()`, como el Store de la práctica 05.

7. Comprobaciones

Qué	Resultado
`ng build` (producción)	Sin errores ni avisos del proyecto
`ng test` (Vitest)	10 tests OK · 9 de los validadores, probados sin DOM: `new FormControl('ABC123', skuValidator(12)).errors` → `{ skuLongitud: { requerida: 12, actual: 6 } }`
`ng lint`	All files pass linting
Prueba en navegador (vista móvil)	Todos los errores, bloqueo durante el guardado, toast, `reset()` y persistencia tras recargar (capturas de la pág. 2)

8. Ejecutar y enlaces

GitHub: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2 → carpeta `07-Formularios-Reactivos-Inventario`
Ejecutar: en `inventario-app/` → `npm install` y `ionic serve` (abre `http://localhost:8100/nuevo-producto`).